INTRODUCTION AUX BASES DE DONNÉES · L2

Du modèle au SQL

Rappels pour le TD4 : créer les tables, puis les interroger

```sql
SELECT * FROM rappels WHERE td = 4;
```

AVANT DE COMMENCER

Trois étapes par exercice

1

Lire les INSERT

Retrouver les tables, leurs colonnes et leurs types. Le MLD et le MCD s'en déduisent comme aux TD précédents.

`INSERT INTO`

2

Créer les tables

Écrire un CREATE TABLE par table, avec sa clé primaire et ses clés étrangères.

`CREATE TABLE`

3

Interroger la base

Répondre aux onze questions par des requêtes SELECT, de la plus simple à la plus fine.

`SELECT`

Les trois exercices ont la même structure : ces rappels servent pour chacun.

AVANT DE COMMENCER

Une base d’exemple : les restos U

restaurant

ref_resto	nom_resto	campus
1	Le Bosquet	Nord
2	Les Pins	Nord
3	La Halle	Sud

Une ligne de **service** : tel jour, tel restaurant a servi tant de portions de tel plat.

plat

ref_plat	nom_plat	prix
1	Curry vert	3.30
2	Lasagnes	4.10
3	Dahl corail	2.90
4	Poulet curry	3.60

jour

ref_jour	date_jour
1	2026-11-09
2	2026-11-10

service

ref_resto	ref_plat	ref_jour	portions
1	1	1	120
1	1	2	80
1	2	1	150
2	1	1	90
2	3	1	60
2	3	2	70

Même forme que dans vos exercices : trois tables, reliées par une quatrième.

ÉTAPE 1 · LIRE LES INSERT

Ce que dit un INSERT INTO

```
INSERT INTO plat (ref_plat, nom_plat, prix)
VALUES (3, 'Dahl corail', 2.90);
```

La table

`plat`

Son nom suit INSERT INTO.

Les colonnes

`(ref_plat, nom_plat, prix)`

Entre parenthèses, après le nom de la table.

Les valeurs

`(3, 'Dahl corail', 2.90)`

Une par colonne, dans le même ordre.

Un INSERT par table suffit pour connaître toutes les tables de la base et leurs colonnes.

ÉTAPE 1 · LIRE LES INSERT

Des valeurs aux types

VALEUR LUE	NATURE	TYPE SQL	À SAVOIR
`3`	**nombre entier**	`INTEGER`	écrit tel quel, sans apostrophes
`2.90`	**nombre décimal**	`NUMERIC(4,2)`	4 chiffres en tout, dont 2 après le point ; jamais de virgule
`'Dahl corail'`	**texte**	`VARCHAR(50)`	entre apostrophes ; 50 caractères au plus
`'2026-11-09'`	**date**	`DATE`	entre apostrophes, au format année-mois-jour
`'12:30'`	**heure**	`TIME`	entre apostrophes, au format heures:minutes

ÉTAPE 2 · CRÉER LES TABLES

CREATE TABLE : une table simple

```
CREATE TABLE plat (
    ref_plat  INTEGER PRIMARY KEY,
    nom_plat  VARCHAR(50) NOT NULL,
    prix      NUMERIC(4,2)
);
```

Les tables restaurant et jour se créent de la même façon.

Une ligne par colonne : son nom, son type, puis ses contraintes.

PRIMARY KEY désigne la clé primaire : une valeur unique et obligatoire. On l'écrit sur la ligne de la colonne quand la clé tient en une seule colonne.

NOT NULL rend la valeur obligatoire.

Ponctuation : une virgule entre deux colonnes, aucune après la dernière, un point-virgule à la fin.

ÉTAPE 2 · CRÉER LES TABLES

CREATE TABLE : la table d’association

```
CREATE TABLE service (
    ref_resto  INTEGER,
    ref_plat   INTEGER,
    ref_jour   INTEGER,
    portions   INTEGER,
    PRIMARY KEY (ref_resto, ref_plat, ref_jour),
    FOREIGN KEY (ref_resto) REFERENCES restaurant(ref_resto),
    FOREIGN KEY (ref_plat)  REFERENCES plat(ref_plat),
    FOREIGN KEY (ref_jour)  REFERENCES jour(ref_jour)
);
```

Clé primaire composée : une seule ligne PRIMARY KEY, après les colonnes, qui les nomme toutes les trois.

Clés étrangères : une ligne FOREIGN KEY par table reliée. Chacune vise la clé primaire de cette table.

Écrire PRIMARY KEY sur chaque colonne est refusé : une table n’a qu’une clé primaire.

ÉTAPE 2 · CRÉER LES TABLES

L’ordre de création et d’insertion

1 · D’ABORD LES TABLES RÉFÉRENCÉES

`restaurant` `plat` `jour`



2 · PUIS CELLE QUI LES RÉFÉRENCE

`service`

Cet ordre vaut pour les **CREATE TABLE** comme pour les **INSERT** : une ligne de service ne peut citer qu’un restaurant, un plat et un jour qui existent déjà.

Sinon PostgreSQL refuse la ligne. Exemple avec un service du restaurant 9, qui n’existe pas :

```
ERROR:  insert or update on table "service" violates foreign key constraint "service_ref_resto_fkey"
DETAIL:  Key (ref_resto)=(9) is not present in table "restaurant".
```

Remède : corriger la valeur, ou insérer d’abord la ligne manquante dans restaurant.

ÉTAPE 3 · INTERROGER UNE TABLE

SELECT … FROM … WHERE

Toute la table

```
SELECT *
FROM plat;
```

ref_plat	nom_plat	prix
1	Curry vert	3.30
2	Lasagnes	4.10
3	Dahl corail	2.90
4	Poulet curry	3.60

Certaines colonnes

```
SELECT nom_plat, prix
FROM plat;
```

nom_plat	prix
Curry vert	3.30
Lasagnes	4.10
Dahl corail	2.90
Poulet curry	3.60

Certaines lignes

```
SELECT nom_plat
FROM plat
WHERE prix < 3;
```

nom_plat
Dahl corail

***** signifie « toutes les colonnes ». **WHERE** ne garde que les lignes qui vérifient la condition.

ÉTAPE 3 · INTERROGER UNE TABLE

Les valeurs dans une condition

Nombre	`WHERE prix < 3`	écrit tel quel
Texte	`WHERE nom_resto = 'Le Bosquet'`	entre apostrophes simples, à la lettre près : majuscules et espaces comptent
Date	`WHERE date_jour = '2026-11-09'`	entre apostrophes, au format année-mois-jour
Comparer	`=` `<>` `<` `⩽` `>` `⩾`	<> signifie « différent de »
Combiner	`AND` `OR` `NOT`	plusieurs conditions dans un même WHERE

⚠ **Piège** : les guillemets doubles désignent une colonne, pas un texte.

`WHERE nom_resto = "Le Bosquet"` → `ERROR: column "Le Bosquet" does not exist`

ÉTAPE 3 · INTERROGER UNE TABLE

DISTINCT : supprimer les doublons

Sans DISTINCT

```
SELECT campus
FROM restaurant;
```

campus
Nord
Nord
Sud

Avec DISTINCT

```
SELECT DISTINCT campus
FROM restaurant;
```

campus
Nord
Sud

DISTINCT s'écrit une seule fois, juste après SELECT. Il compare des lignes entières : avec deux colonnes affichées, seules les lignes identiques sur les deux sont fusionnées.

ÉTAPE 3 · INTERROGER UNE TABLE

LIKE : chercher un motif dans un texte

```
SELECT nom_plat
FROM plat
WHERE nom_plat LIKE '%curry%';
```

RÉSULTAT

nom_plat
Poulet curry

`%`	n'importe quelle suite de caractères, même vide
`_`	exactement un caractère
`'Cur%'`	commence par « Cur »
`'%ail'`	se termine par « ail »
`'%curry%'`	contient « curry »

⚠ **Piège** : LIKE distingue majuscules et minuscules, donc « Curry vert » manque. Pour l'obtenir aussi, sous PostgreSQL :

```
nom_plat ILIKE '%curry%'   ou   LOWER(nom_plat) LIKE '%curry%'
```

ÉTAPE 3 · CROISER LES TABLES

Une jointure relie une clé étrangère à la clé primaire qu'elle référence.

```
service.ref_plat = plat.ref_plat
```

ÉTAPE 3 · CROISER LES TABLES

JOIN … ON : relier deux tables

```
SELECT r.nom_resto, s.portions
FROM service s
JOIN restaurant r ON r.ref_resto = s.ref_resto;
```

s et **r** sont des alias : ils abrègent les noms des tables et préfixent leurs colonnes.

ON porte la condition de jointure : clé étrangère = clé primaire.

Autre écriture, même résultat :

```
FROM service s, restaurant r
WHERE r.ref_resto = s.ref_resto;
```

RÉSULTAT

nom_resto	portions
Le Bosquet	120
Le Bosquet	80
Le Bosquet	150
Les Pins	90
Les Pins	60
Les Pins	70

Chaque service est complété par le nom de son restaurant.

ÉTAPE 3 · CROISER LES TABLES

Plusieurs jointures à la suite

```sql
SELECT p.nom_plat
FROM service s
JOIN restaurant r ON r.ref_resto = s.ref_resto
JOIN plat p       ON p.ref_plat  = s.ref_plat
JOIN jour j       ON j.ref_jour  = s.ref_jour
WHERE r.nom_resto = 'Le Bosquet'
  AND j.date_jour = '2026-11-09';
```

RÉSULTAT

nom_plat
Curry vert
Lasagnes

LA MÉTHODE

1. Quelles colonnes afficher ? Repérez leurs tables.
2. Quelles conditions poser ? Repérez leurs tables.
3. Partez de la table d'association et ajoutez un JOIN par table repérée, pas davantage.

Ici : nom_plat vient de plat, les conditions portent sur restaurant et jour, et service relie les trois.

ÉTAPE 3 · CROISER LES TABLES

Deux pièges des jointures

Colonne ambiguë

ref_plat existe dans service et dans plat. Sans préfixe, PostgreSQL ne sait pas laquelle choisir :

```
ERROR:  column reference "ref_plat"
is ambiguous
```

Remède : préfixer la colonne, s.ref_plat ou p.ref_plat.

Lignes en double

Les plats servis aux Pins : le dahl y a été servi deux jours, il sort donc deux fois.

nom_plat
Curry vert
Dahl corail
Dahl corail

Remède : écrire SELECT DISTINCT pour ne garder chaque plat qu'une fois.

ÉTAPE 3 · CALCULER ET REGROUPER

Une colonne calculée, un alias

```
SELECT s.ref_resto, s.ref_plat, s.ref_jour,
       s.portions * p.prix AS recette
FROM service s
JOIN plat p ON p.ref_plat = s.ref_plat;
```

Le calcul se fait **ligne par ligne**, avec +, −, * ou /.

AS donne un nom à la colonne calculée.

portions est dans service, **prix** dans plat : le calcul exige la jointure.

RÉSULTAT

ref_resto	ref_plat	ref_jour	recette
1	1	1	396.00
1	1	2	264.00
1	2	1	615.00
2	1	1	297.00
2	3	1	174.00
2	3	2	203.00

Première ligne : 120 portions × 3.30 = 396.00.

ÉTAPE 3 · CALCULER ET REGROUPER

SUM et GROUP BY : un total par groupe

```
SELECT ref_resto, ref_plat, SUM(portions) AS total
FROM service
GROUP BY ref_resto, ref_plat;
```

GROUP BY réunit les lignes de même restaurant et de même plat ; **SUM** additionne leurs portions.

Autres fonctions : COUNT, AVG, MIN, MAX.

service

ref_resto	ref_plat	ref_jour	portions
1	1	1	120
1	1	2	80
1	2	1	150
2	1	1	90
2	3	1	60
2	3	2	70



RÉSULTAT

ref_resto	ref_plat	total
1	1	200
1	2	150
2	1	90
2	3	130

Chaque teinte marque un groupe : un couple restaurant-plat.

Premier groupe : deux lignes, 120 + 80 = 200.

ÉTAPE 3 · CALCULER ET REGROUPER

La règle du GROUP BY

Dans le SELECT, toute colonne qui n'est pas dans une fonction d'agrégation doit figurer dans le GROUP BY.

REFUSÉ

```
SELECT ref_resto, ref_plat, SUM(portions)
FROM service
GROUP BY ref_resto;
```

```
ERROR:  column "service.ref_plat" must appear
in the GROUP BY clause or be used in an
aggregate function
```

ACCEPTÉ

```
SELECT ref_resto, ref_plat, SUM(portions)
FROM service
GROUP BY ref_resto, ref_plat;
```

Le groupe du restaurant 1 contient deux plats : PostgreSQL ne peut pas en afficher un seul. Regroupez aussi par plat, ou retirez ref_plat du SELECT.

ÉTAPE 3 · CHERCHER L'ABSENCE

NOT IN : ce qui n'apparaît nulle part

Quels plats n'ont jamais été servis ?

```
SELECT nom_plat
FROM plat
WHERE ref_plat NOT IN (SELECT ref_plat FROM service);
```

1. La sous-requête, entre parenthèses, donne les plats servis : 1, 1, 2, 1, 3, 3.
2. **NOT IN** garde les plats dont la référence n'est pas dans cette liste : le plat 4.

RÉSULTAT

nom_plat
Poulet curry

Une jointure seule ne répond pas : elle ne garde que les plats qui ont au moins un service.

ÉTAPE 3 · CHERCHER L'ABSENCE

LEFT JOIN … IS NULL : l'autre écriture

```sql
SELECT p.nom_plat
FROM plat p
LEFT JOIN service s ON s.ref_plat = p.ref_plat
WHERE s.ref_plat IS NULL;
```

LEFT JOIN garde tous les plats, même sans service : les colonnes venues de service valent alors NULL.

IS NULL repère ces lignes. N'écrivez jamais = NULL : cette comparaison n'est jamais vraie.

APRÈS LE LEFT JOIN, AVANT LE WHERE

p.nom_plat	s.ref_plat	s.portions
Curry vert	1	120
Curry vert	1	80
Curry vert	1	90
Lasagnes	2	150
Dahl corail	3	60
Dahl corail	3	70
Poulet curry	NULL	NULL

Seule la dernière ligne passe le filtre IS NULL.

À GARDER SOUS LES YEUX

L’ordre des clauses

SELECT	`colonnes, calcul AS nom, SUM(…)`	Que faut-il afficher ?
FROM	`table`	Dans quelle table chercher ?
JOIN	`table ON clé étrangère = clé primaire`	Quelles autres tables relier ?
WHERE	`condition sur les lignes`	Quelles lignes garder ?
GROUP BY	`colonnes de regroupement`	Un total par quoi ?

Toujours dans cet ordre ; les clauses inutiles s’omettent. DISTINCT se place juste après SELECT.